

به نام خدا



دانشگاه تهران



دانشکده مهندسی برق و کامپیوتر

درس یادگیری تقویتی

تمرین چهارم

نام و نام خانوادگی	امیر غرقابی
شماره دانشجویی	۸۱۰۱۰۲۲۱۷
تاریخ ارسال گزارش	۱۴۰۳.۱۰.۲۲

فهرست

۱. مقایسه روش های مبتنی بر ارزش و سیاست و ترکیب آن ها ۱ پاسخ
۲. Dueling DQN ۲ پاسخ
۳. A2C ۳ پاسخ
۴. محیط Cart Pole – v1 ۵ پاسخ
۵. چرا Classic RL برای این محیط خوب نیست؟ ۶ پاسخ
۶. پیاده سازی DQN در محیط Cartpole ۶ پاسخ
۷. پیاده سازی Dueling DQN در محیط Cartpole ۸ پاسخ
۸. مقایسه DQN و Dueling DQN در محیط Cartpole ۱۰ پاسخ
۹. پیاده سازی A2C ۱۱ پاسخ
۱۰. مقایسه DQN و Dueling DQN در محیط Cartpole ۱۳ پاسخ

شکل‌ها

- شکل ۱. شبکه‌های Q-Learning (بالا) و Dueling Q-learning (پایین) ۲
- شکل ۲. تصویری با جزئیات بیشتر از روش Dueling DQN ۲
- شکل ۳. ساختار شبکه A2C ۴
- شکل ۴. شمایی از محیط cart pole- v1 ۵
- شکل ۵. نمودار پاداش cartpole در الگوریتم DQN ۷
- شکل ۶. نمودار پاداش cartpole در الگوریتم Dueling DQN ۸
- شکل ۷. مقایسه همگرایی DQN و Dueling DQN ۱۰
- شکل ۸. نمودار پاداش cartpole در الگوریتم A2C ۱۲

پاسخ ۱. مقایسه روش های مبتنی بر ارزش و سیاست و ترکیب آن ها

مقایسه یادگیری **value based** و **policy based**:

روش های مبتنی بر سیاست، به طور طبیعی مصالحه میان Exploration exploitation را برقرار می کنند اما روش های مبتنی بر value، نیازمند روشی برای اکتشاف هستند مانند epsilon greedy.

معمولا روش های مبتنی بر سیاست در فضاهای پیوسته کارآمدتر هستند. اما واریانس بالاتری دارند. همچنین روش های مبتنی بر سیاست معمولا به تعداد داده بیشتری نیاز دارند.

همچنین اگر سیاست بهینه، stochastic باشد، استفاده از روش های مبتنی بر ارزش عملکرد خوبی نخواهند داشت و روش های مبتنی بر سیاست مناسبتر هستند. [1]

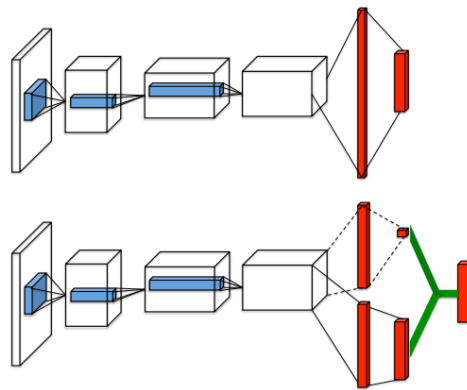
در بسیاری از کاربردها، مثلا در محیط های multi agent، عامل می تواند رفتار سایر عامل ها را ببیند (actions) اما اطلاعی از پاداشی که دریافت کرده اند ندارند. بنابراین نمیتوانند به طور مستقیم از روش های value based استفاده کنند و روش های مبتنی بر سیاست بهتر عمل خواهند کرد.

آیا روشی وجود دارد که از هر دو رویکرد استفاده کند؟

بله، روش هایی به نام actor critic وجود دارند که از ترکیب دو متد به وجود آمده و از نقاط قوت هر دو روش استفاده می کنند. در این روش actor (سیاست) مسئول انتخاب action بر اساس سیاست فعلی است (که ممکن است stochastic باشد) و critic (ارزش) وظیفه ی تخمین تابع ارزش را دارد. سپس از سیاست actor بر اساس بازخوردی که از critic می گیرد، آپدیت می شود و actor سعی می کند که action هایی را انتخاب کند که منجر به پاداش های بیشتری شود. actor و critic با روش های مبتنی بر گرادیان آموزش دیده و آپدیت می شوند. روش actor-critic پایداری بهتری از روش های مبتنی بر سیاست و روش های مبتنی بر ارزش دارند همچنین برای مسائلی با state و action با ابعاد بالا مناسب هستند. [2]

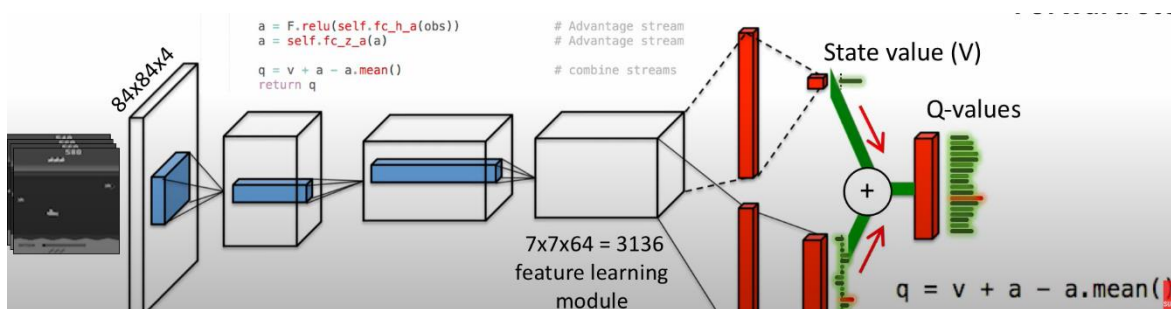
پاسخ ۲. Dueling DQN

در شکل ۱، شمایی از تفاوت شبکه‌ی عصبی در دو الگوریتم Q-learning و dueling q-learning قابل مشاهده است [3].



شکل ۱. شبکه‌های Q-Learning (بالا) و Dueling Q-learning (پایین)

در شکل ۲، به صورت جزئی تر، روابط Dueling DQN نشان داده شده است. (مرجع تصویر [4])



شکل ۲. تصویری با جزئیات بیشتر از روش Dueling DQN

شبکه double q-learning، دارای دو تخمین زن است (به بیان دیگر ساختار به نسبت پیچیده تری نسبت به q-learning دارد) که یکی برای تخمین state-value function است (که این تخمین زن در شبکه q-learning نیز وجود دارد) و دیگری برای تخمین تابع advantage اکشن هایی است که وابسته به state هستند. درواقع advantage function نشان می‌دهد که اکشن a_i در استیت S_j چقدر از سایر اکشن ها بهتر (یا بدتر) است.

الگوریتم Dueling DQN شامل دو دسته خروجی در لایه یکی مانده به آخر است.

- Value stream: خروجی این بخش، تنها شامل یک مقدار یعنی state value است.
- Advantage stream: این بخش به تعداد action ها خروجی دارد. رابطه ی advantage function بصورت زیر است.

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$$

سپس این دو لایه به صورت زیر با یکدیگر ترکیب شده و q-value متناظر با هر اکشن را در خروجی تولید می کنند:

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + \left(A(s, a; \theta, \alpha) - \max_{a' \in |\mathcal{A}|} A(s, a'; \theta, \alpha) \right).$$

البته در ادامه مقاله آمده است که می توان به جای اپراتور max، از میانگین نیز استفاده کرد یعنی: [2.2]

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + \left(A(s, a; \theta, \alpha) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'; \theta, \alpha) \right).$$

پاسخ ۳. A2C

در مقاله [3.1] روش های asynchronous برای الگوریتم های Actor Critic و (off policy) q-learning (on policy) که برای تخمین سیاستهای stochastic، به خوبی عمل می کند، توضیح داده شده است. این روش شامل ۳ جز اصلی است.

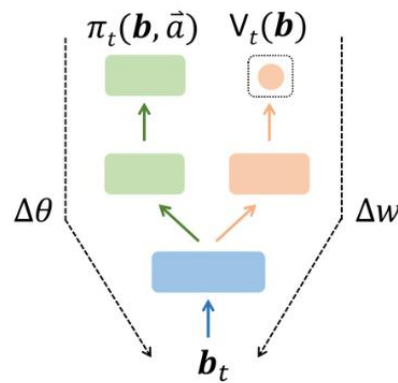
- Policy: یادگیری سیاست.
- Critic: ارزیابی سیاست بر اساس value function
- Advantage function: رابطه ی $A(s_t, a_t; \theta, \theta_v)$ که با استفاده از رابطه زیر محاسبه می شود.

$$\begin{aligned} A(s_t, a_t; \theta, \theta_v) &= R_t(s_t) - V(s_t, \theta_v) \\ &= \sum_{i=0}^{k-1} \gamma^i r_{t+i} + \gamma^k V(s_{t+k}; \theta_v) - V(s_t; \theta_v) \end{aligned}$$

در روش های asynchronous، از چند thread در cpu یک دستگاه بصورت موازی استفاده می شود. مزیت آن، این است که هزینه های تبادل اطلاعات (مانند گرادیان ها و پارامترها) میان عامل ها از بین می رود. البته روش A2C نسخه ساده شده ی A3C است که تنها از یک عامل استفاده می کند.

تفاوت های A2C و Dueling DQN بصورت زیر است.

- روش A2C بر خلاف روش های DQN و Dueling DQN از replay buffer استفاده نمی کند.
 - روش A2C، بصورت On-policy عمل میکند در حالیکه روش های DQN و Dueling DQN بصورت off-policy عمل می کنند.
 - تابع advantage function در روش A2C، بصورت $A(s_t, a_t; \theta, \theta_v) = R_t(s_t) - V(s_t, \theta_v)$ است در حالیکه در روش Dueling DQN بصورت $A(s_t, a_t; \theta, \theta_v) = Q(s_t, a_t) - V(s_t, \theta_v)$ است.
- در روش A2C، بطور معمول از یک شبکه CNN به همراه یک لایه softmax برای تخمین سیاست $\pi(a_t | s_t; \theta)$ استفاده می شود و یک لایه خطی نیز برای تخمین تابع ارزش $V(s_t; \theta)$ استفاده می شود. سایر لایه ها میان این دو خروجی یکسان است. تصویری از ساختار این شبکه در شکل ۳ قابل مشاهده است. این تصویر از مقاله ی [3.1] گرفته شده است.



شکل ۳. ساختار شبکه A2C

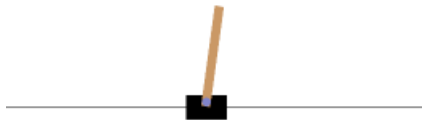
در این الگوریتم، برای اپدیت پارامترها از رابطه ی

$$\nabla_{\theta'} \log \pi(a_t | s_t; \theta') A(s_t, a_t; \theta, \theta_v)$$

استفاده می شود.

پاسخ ۴. محیط Cart Pole – v1

محیط Cart pole، همان مسئله پاندول معکوس در تئوری کنترل است که یک ارا به روی زمین در دو بعد حرکت کرده و سعی میکند یک میله را روی خود پایدار نگه دارد تا میله با زمین برخورد نکند/نیافتد. تصویری از این محیط در شکل ۴ قابل مشاهده است.



شکل ۴. شمایی از محیط cart pole- v1

فضای action و state بصورت زیر است.

- فضای action به صورت گسسته و دو بعدی است.
 - 0: حرکت ارا به چپ
 - 1: حرکت ارا به راست
- فضای state: ۴ بعدی است. همه ابعاد آن پیوسته هستند.
 - موقعیت ارا به: عددی پیوسته در بازه ی $[-4.8, +4.8]$. اگر ارا به از بازه ی $[-2.4, +2.4]$ خارج شود، اپیزود terminate می شود.
 - سرعت ارا به: عددی پیوسته در بازه ی $[-inf, +inf]$
 - زاویه میله: عددی پیوسته در بازه ی $[-0.418, +0.418]$. اگر زاویه میله از بازه ی $[-0.2095, +0.2095]$ خارج شود، اپیزود terminate می شود.
 - سرعت زاویه ای میله: عددی پیوسته در بازه ی $[-inf, +inf]$
- Reward: در هر step که میله زمین نخورد پاداش +1 دریافت می کنند. (بیشترین پاداش به علت محدودیت ها، می تواند 500 باشد). اگر هم اپیزود terminate شود، پاداش -1 دریافت می شود. اگر هم Sutton_barto_reward=True باشد، به ازای هر step که تسک terminate نشود، میزان پاداش برابر 0 خواهد بود.

هر اپیزود در ۲ حالت terminate می شود که در بخش فضای state معرفی شد. همچنین اگر طول اپیزود بیشتر از 500 باشد نیز، truncate می شود. یعنی بیشترین پاداشی که عامل می تواند در یک اپیزود بگیرد، ۵۰۰ است. به بیان دیگر اگر میله به خوبی کنترل شود، نهایتاً ۵۰۰ time step اپیزود ادامه خواهد داشت.

پاسخ 5. چرا Classic RL برای این محیط خوب نیست؟

روش Q-learning کلاسیک، برای مسائلی که ابعاد state و یا action محدود است مناسب هستند. اما در مسئله Cart pole، فضای state پیوسته است. البته میتوان این فضای پیوسته را، گسسته سازی کرد اما در این حالت هم تعداد state ها بسیار زیاد بوده محاسبات بسیار دشوار و پیچیده می کند و به فضای زیادی در حافظه کامپیوتر نیاز خواهد داشت و البته روش های کلاسیک RL، تضمینی برای تعمیم پذیری در state هایی که ندیده اند ندارند. [5].

پاسخ 6. پیاده سازی DQN در محیط Cartpole

در این بخش با استفاده از Deep Q Learning مسئله ی پاندول معکوس (cart pole) حل می شود. درواقع میله متصل به ارابه، کنترل می شود که ناپایدار نشود. ارابه باید بین دو action یعنی حرکت به چپ و راست انتخاب نماید تا میله نیافتد. هرچه میله به مدت طولانی تری ثابت بماند، عامل پاداش بیشتری دریافت خواهد کرد.

شبکه ی Q-network: این شبکه در کد با نام DQN پیاده سازی شده است. ورودی این شبکه state بوده و خروجی آن، q-value اکشن های راست و چپ است. این شبکه یک لایه ورودی، دو لایه میانی و یک لایه خروجی دارد. [6][7]

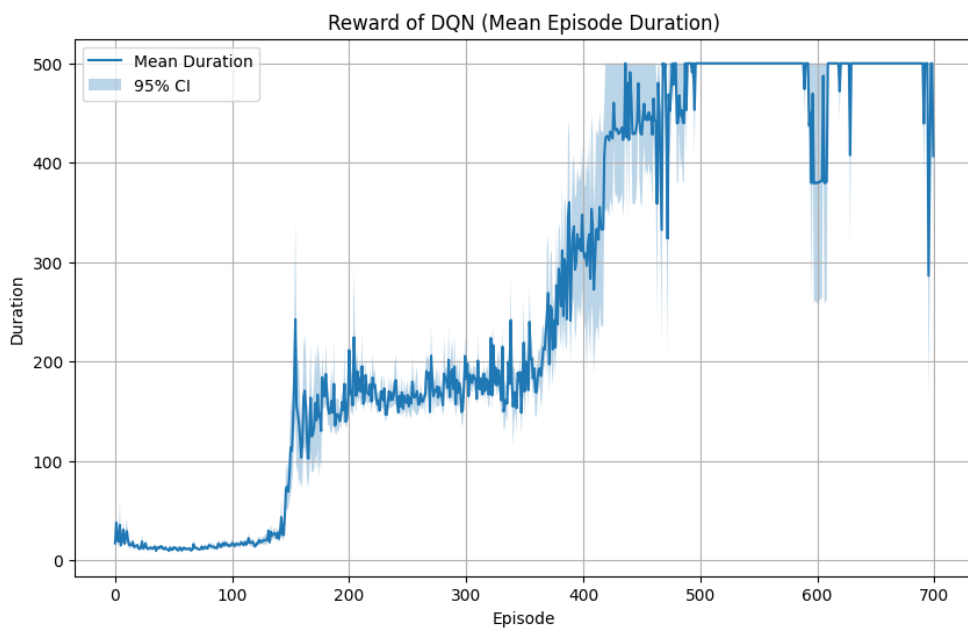
پارامترهای استفاده شده در جدول ۱ قابل مشاهده هستند.

جدول ۱. هایپرپارامترهای استفاده شده در الگوریتم DQN

پارامتر	مقدار
ظرفیت buffer	10000
سایز هر batch	128
discount factor γ	0.99
ϵ_{init}	0.9
ϵ_{min}	0.05
Learning rate α	0.0001
τ نرخ اپدیت شبکه target	0.05
optimizer	Adam

شبکه ی target به صورت soft و با پارامتر τ اپدیت می شود.

الگوریتم در هر تکرار ۷۰۰ اپیزود را اجرا می کند. ۵ بار ران شده و نتیجه نمودار reward (که در اینجا مدت زمان پایداری میله است) در شکل ۵ نشان داده شده است. همانطور که قابل مشاهده است، پس از حدود ۴۵۰ اپیزود، میزان پاداش به ۵۰۰ همگرا شده است. در هر اپیزود، پاداش بیشتر از ۵۰۰ نیز ممکن نیست زیرا در محیط cartpole در کتابخانه gym، الگوریتم بعد از ۵۰۰ step، truncate می شود.



شکل ۵. نمودار پاداش cartpole در الگوریتم DQN

همچنین در تکرار آخر، به ازای هر ۱۵۰ اپیزود یک ویدیو از عملکرد مدل ضبط شده است که در آدرس DQN-videos قابل مشاهده هستند. مجموعاً ۵ ویدیو در اپیزودهای ۱، ۱۰۰، ۴۰۰، ۵۰۰ و ۷۰۰ مشخص است که هرچه شماره اپیزود بالاتر باشد، میله پایدارتر خواهد بود چون طبق شکل فوق، در اپیزودهای بالا، میزان reward به ۵۰۰ همگرا شده است. یعنی میله در ۵۰۰ فریم نیافتاده است. همچنین در تولید ویدیوها، $\text{fps}=30$ در نظر گرفته شده است بنابراین مدت زمان ویدیو در بیشترین مقدار خود، باید $16^s = \frac{500}{30}$ ثانیه باشد که در ویدیوی آخر همین اتفاق افتاده است و میله به مدت ۱۶ ثانیه ثابت مانده است.

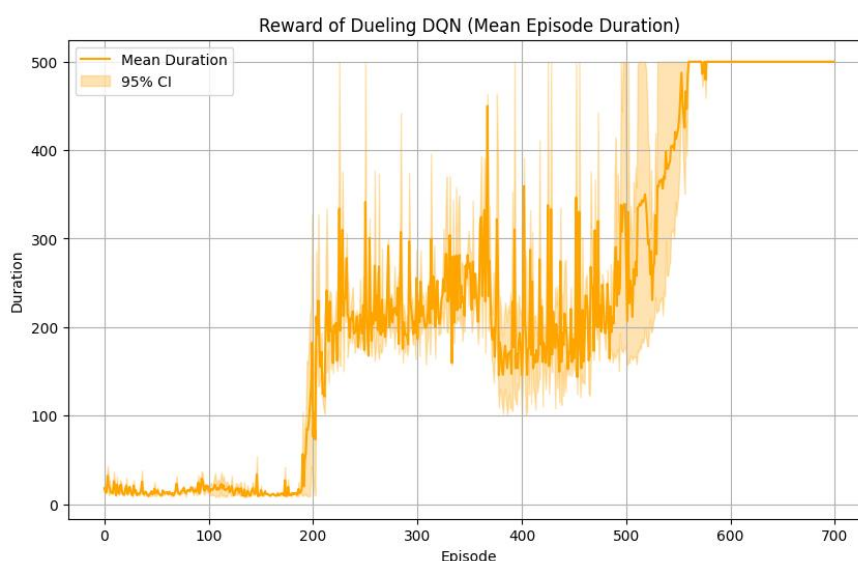
پاسخ ۷. پیاده سازی Dueling DQN در محیط Cartpole

در این بخش با استفاده از Dueling Deep Q Learning مسئله ی پاندول معکوس (cart pole) حل می شود. تفاوت Q-learning و Dueling Q-learning از نظر ساختار شبکه عصبی، در سوال ۲ بررسی شد. درواقع یک stream در شبکه اضافه شده است که محاسبه ی زیر را انجام داده و مقادیر q-value را به ازای هر اکشن خروجی می دهد. [8]

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + \left(A(s, a; \theta, \alpha) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'; \theta, \alpha) \right).$$

الگوریتم در ۴ ران و هر ران شامل ۵۰۰ اپیزود اجرا شده و نتیجه در شکل ۶ قابل مشاهده است.

توجه: پیاده سازی الگوریتم های DQN و Dueling DQN هر دو در فایل dqn.ipynb قرار دارد.



شکل ۶: نمودار پاداش cartpole در الگوریتم Dueling DQN

مشاهده می شود که این الگوریتم بعد از حدود ۵۰۰ اپیزود همگرا شده است. یعنی در نهایت، ارا به توانسته است میله در بازه ی امن state ها نگه دارد (هم از نظر زاویه میله و هم محل قرار گیری ارا به) پارامترهای استفاده شده در جدول ۲ قابل مشاهده هستند.

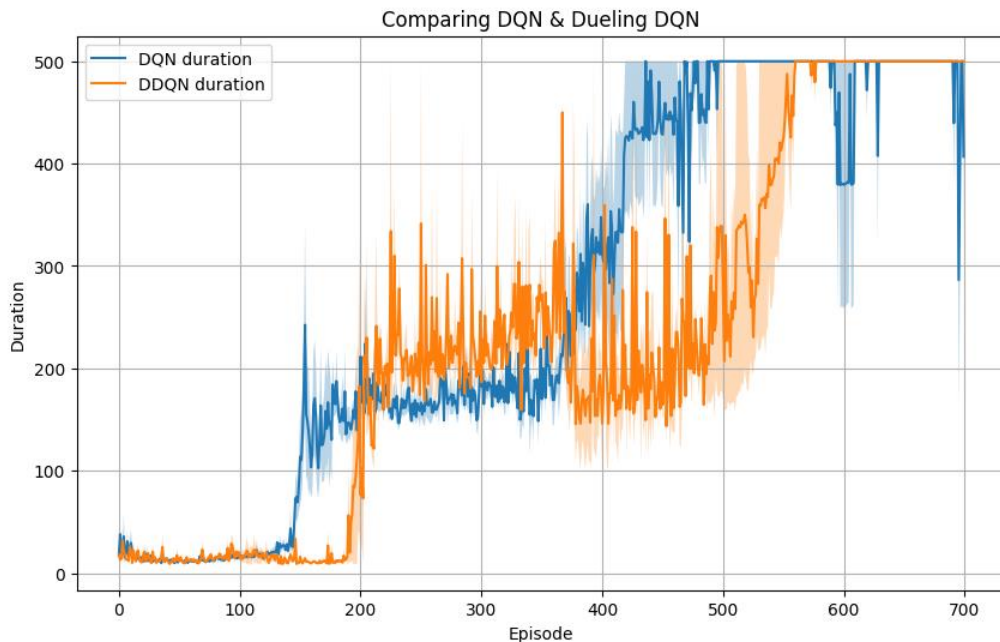
جدول ۲. هایپرپارامترهای استفاده شده در الگوریتم DQN

پارامتر	مقدار
ظرفیت buffer	10000
سایز هر batch	128
discount factor γ	0.99
ϵ_{init}	0.9
ϵ_{min}	0.05
Learning rate α	0.0001
τ نرخ اپدیت شبکه target	0.05
optimizer	Adam

ویدیوهای عملکرد عامل در محیط cartpole در اپیزودهای 1, 100, 400, 700 در پوشه ی DDQN-videos قابل مشاهده است.

پاسخ ۸. مقایسه DQN و Dueling DQN در محیط Cartpole

در این بخش نمودار همگرایی هر دو را رسم کرده و مقایسه می کنیم.



شکل ۷. مقایسه همگرایی DQN و Dueling DQN

هر دو الگوریتم به خوبی عمل کرده و به پاداش 500 همگرا شده اند. هر دو نمودار به صورت stochastic تولید شده اند. اگرچه در این نمودار، روش DQN عملکرد بهتری داشته است، اما چون هر دو نمودار به صورت stochastic رسم می شوند، ممکن است در اجراهای بعدی، شاهد این باشیم که Dueling DQN عملکرد بهتری از خود نشان داده است. به علت محدودیت های سخت افزاری، تعداد ران هر الگوریتم تنها ۲ بار است و اگر این تعداد به ۲۰ افزایش یابد، می توان مقایسه بهتری داشت و انتظار می رود که در نهایت، الگوریتم Dueling DQN کمی بهتر از DQN عمل کند.

الگوریتم Dueling DQN ساختار به نسبت پیچیده تری داشته و بار محاسباتی آن به نسبت DQN کمی بیشتر است. این الگوریتم اگر برخی از اکشن ها اهمیت بیشتری نسبت به سایر اکشن ها داشته باشد، به خوبی عمل می کند. اما در مسئله cartpole که نسبتا مسئله ساده ای به شمار می رود که هر دو اکشن (right, left) از اهمیت یکسانی برخوردار هستند، بنظر میرسد الگوریتم DQN انتخاب مناسبی برای حل باشد.

همچنین به دلیل نسبتا ساده بودن محیط، یکی از مزایای Dueling DQN یعنی کاهش overestimation bias، خود را به خوبی نشان نمیدهد.

عامل دیگر تاثیر گذار در سرعت همگرایی این دو الگوریتم، تنظیم مناسب هایپرپارامترها است. در این مسئله هایپرپارامتر هر دو الگوریتم یکسان در نظر گرفته شده اند. ممکن است با انتخاب پارامترهای بهتر، نتیجه تغییر کند.

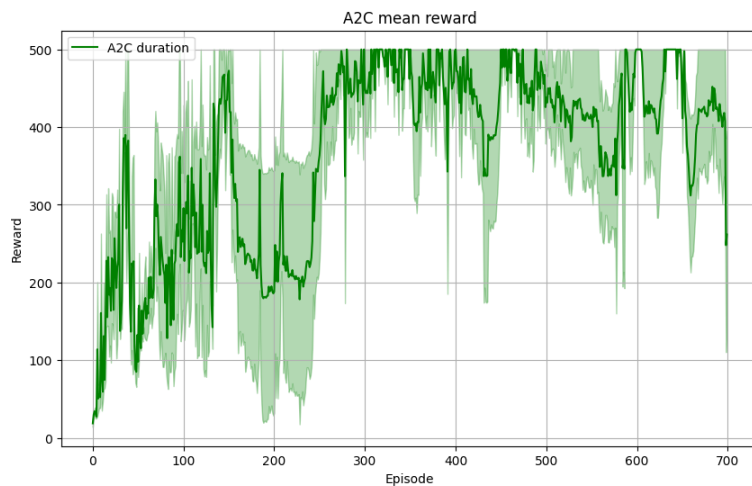
پاسخ ۹. پیاده سازی A2C

در این بخش از کتابخانه stable baseline برای یادگیری محیط cartpole استفاده می شود. این کتابخانه نسخه ی synchronous الگوریتم A3C است. درواقع روش A2C نسخه ی ساده شده ی A3C است که تنها از یک عامل برای یادگیری استفاده می کند. اما با اینکه از تنها یک عامل استفاده میکند، همچنان به replay buffer نیازی ندارد. روش A2C یک الگوریتم On-policy است. پارامترهای استفاده شده در این بخش، همان پارامترهای دیفالت کتابخانه است. [9]

جدول ۳. هایپرپارامترهای استفاده شده در الگوریتم A2C

پارامتر	مقدار
batch	200
discount factor γ	0.99
Learning rate α	0.0007
تعداد محیط n_environment	1

نتیجه شبیه سازی برای ۵ ران، و هر ران ۷۰۰ اپیزود در شکل ۸ قابل مشاهده است.



شکل ۸. نمودار پاداش **cartpole** در الگوریتم **A2C**

همانطور که مشاهده می شود، میزان پاداش به عدد ۵۰۰ همگرا شده است. همچنین ویدیوهای عملکرد عامل در محیط **cartpole** در اپیزودهای 1, 100, 400, 700 در پوشه ی **A2C-videos** قابل مشاهده است. همانطور که در جدول ۳ نیز مشخص است، در هر اپیزود، عامل با مجموعه داده ای به اندازه ی **batch_size=200**، تابع **learn** را اجرا کرده و پارامترهای خود را آپدیت می کند.

الگوریتم **A2C** در طراحی ر نظر گرفته شده، در هر اپیزود به 200 نمونه (batch size) نیاز داشته است. که در نهایت در اپیزود 300، مجموعاً 60.000 داده می شود.

پاسخ ۱۰. مقایسه DQN و Dueling DQN در محیط Cartpole

در این بخش به مقایسه نتایج A2C و DQN می پردازیم.

در واقع شکل های ۸ و ۵ با یکدیگر مقایسه می شوند.

همانطور که مشخص است الگوریتم A2C در حدود ۳۰۰ اپیزود همگرا شده است. در حالیکه هر دو الگوریتم DQN و Dueling DQN در حدود ۵۰۰ اپیزود همگرا شده اند. البته در الگوریتم DQN، شاهد پایداری به نسبت بیشتری هستیم یعنی عامل پس از همگرا شدن به پاداش ۵۰۰، این پاداش را حفظ کرده است در حالیکه در A2C در اپیزودهای بالاتر از 300، شاهد نوسان در میزان پاداش و واریانس نسبتا بالاتر هستیم. در کل میزان واریانس در روش A2C به نسبت بیشتر از روش DQN بوده است.

الگوریتم A2C ساختاری به نسبت پیچیده تر نسبت به DQN دارد. در واقع ترکیبی از دو روش مبتنی بر ارزش و مبتنی بر سیاست است.

در ادامه روش های A2C و DQN مقایسه می شوند. (لازم به ذکر است که این مقایسه با توجه به نمودارهای بدست آمده انجام شده است.)

- مقایسه سرعت (از نظر تعداد اپیزود): $A2C > DQN$ (روش A2C در تعداد اپیزود کمتری همگرا شد)
- مقایسه سرعت (از نظر زمان اجرای برنامه): $A2C > DQN$ (روش A2C بسیار سریعتر پاسخ بهینه را پیدا می کرد)
- مقایسه پایداری: $DQN > A2C$
- دقت در محاسبه مقدار q-value ها: $Dueling Q > A2C$

در این مسئله که فضای اکشن، گسسته است، روش DQN نیز روش مناسبی است. در واقع اگر فضای اکشن ها نیز پیوسته بودند، روش A2C که از policy gradient نیز استفاده می کند، می تواند به مراتب عملکرد بهتری از خود نشان دهد.

البته اگر محدودیت سخت افزاری و زمانی نبود و میتوانستیم هر الگوریتم را در تعداد دفعات بیشتری (مثلا ۳۰ بار) ران کنیم، میتوانستیم مقایسه بهتری با توجه به نمودارها داشته باشیم.

مراجع:

- [1] GPT 4-o, Prompt: “in reinforcement learning, when we can learn the value, why policy based approaches”
- [2] Inspired by GPT 4-o, Prompt: “explain actor critic method”
- [3] <https://arxiv.org/abs/1511.06581v3>
- [4] https://www.youtube.com/watch?v=XjsY8-P4WHM&ab_channel=AndrewMelnik
- [5] Chatgpt prompt: “why we cant use classic RL in cartpole”
- [6] <https://pytorch.org/>
- [7] Inspired by GPT 4-o, Prompt: “code debugging, Q_network construction”
- [8] Chatgpt prompt: “this is my dqn: class DQN(nn.Module) change it to dueling q-learning”
- [9] https://stable-baselines3.readthedocs.io/en/master/_modules/stable_baselines3/a2c/a2c.html#A2C